

Программирование на С

Лекция 12

Строки и файлы

Строки в языке C

Строка в языке C — это массив символов, заканчивающийся специальным символом `'\0'`.

```
char str[] = "Hello";
```

Эквивалентная запись:

```
char str[] = { 'H', 'e', 'l', 'l', 'o', '\0' };
```

Символ конца строки

```
char str[] = "Hello";
```

	0	1	2	3	4	5
str	Н	е	л	л	о	\0

Символ `'\0'` не отображается на экране,
но обозначает конец строки.

Почему нужен `\0`?

Большинство строковых функций не знает размер массива.

Например:

```
puts(str);
```

или

```
strlen(str);
```

Они читают символы подряд,
пока не встретят `'\0'`

Строка — это обычный массив

```
char str[] = "Hello";
```

Можно обращаться к символам по индексу.

```
printf("%c\n", str[1]);
```

e

Можно изменять отдельные символы.

```
str[0] = 'h';
```

Изменение строки

```
char str[] = "Hello";
```

До изменения:

Hello

```
str[0] = 'h';
```

После изменения:

hello

Два способа объявления строки

Строку можно объявить двумя способами.

Массив символов:

```
char str[] = "Hello";
```

Указатель на строковый литерал:

```
const char *str = "Hello";
```

На первый взгляд они очень похожи,
но работают по-разному.

Способ 1. Символьный массив

```
char str[] = "Hello";
```

str



Н	е	л	л	о	\0
---	---	---	---	---	----

Символы являются частью массива.

Можно изменять символы

```
char str[] = "Hello";
```

```
str[0] = 'h';
```

Получится

```
hello
```

Но изменить сам массив нельзя.

```
str = "World";      // ошибка
```

Способ 2. Указатель

```
const char *str = "Hello";
```

str



H	e	l	l	o	\0
---	---	---	---	---	----

Переменная хранит только адрес строки.

Можно изменить указатель

```
const char *str = "Hello";
```

```
str = "World";
```

Теперь

str



World

Указатель начал ссылаться на другую строку.

Нельзя изменять строковый литерал

```
const char *str = "Hello";
```

Попытка изменить символ

```
str[0] = 'h';
```

приведёт к ошибке.

Строковые литералы считаются неизменяемыми.

Именно поэтому используется тип

```
const char *
```

Сравнение двух способов

	<code>char str[]</code>	<code>const char *str</code>
Изменять символы	✓	✗
Изменять переменную	✗	✓
Хранит символы	✓	✗
Хранит адрес	✗	✓

Почему строки нельзя присваивать?

Пусть есть две строки.

```
char a[20] = "Hello";  
char b[20] = "World";
```

Можно ли написать

```
a = b;
```

Ответ

```
char a[20] = "Hello";  
char b[20] = "World";  
  
a = b;      // ошибка
```

Массивы нельзя присваивать друг другу.

Необходимо копировать каждый символ отдельно.

Как копировать строку?

Можно написать цикл.

```
int i = 0;

while (b[i] != '\0')
{
    a[i] = b[i];
    i++;
}

a[i] = '\0';
```

Но такую задачу приходится решать очень часто.

Библиотека string.h

Для работы со строками существует библиотека

```
#include <string.h>
```

Она содержит готовые функции:

- определение длины;
- копирование;
- объединение строк;
- сравнение;
- поиск.

Основные функции

Функция	Назначение
<code>strlen</code>	длина строки
<code>strcpy</code>	копирование
<code>strcat</code>	объединение
<code>strcmp</code>	сравнение
<code>strchr</code>	поиск символа
<code>strstr</code>	поиск подстроки

Длина строки

```
size_t strlen(const char *str);
```

Возвращает количество символов
до первого символа

```
'\0'
```

Пример strlen

```
char str[] = "Hello";  
printf("%zu\n", strlen(str));
```

5

Символ `'\0'` не входит в длину строки.

Копирование строки

```
char *strcpy(char *dest,  
             const char *src);
```

Копирует строку

src

В

dest

Пример strcpy

```
char src[] = "Hello";  
char dst[20];  
  
strcpy(dst, src);  
  
puts(dst);
```

Hello

Что делает strcpy?

До копирования

src:

H e l l o \0

dst:

? ? ? ? ? ? ?

После копирования

src:

H e l l o \0

dst:

H e l l o \0

Опасность strcpy

```
char dst[5];  
  
strcpy(dst,  
        "Hello, world!");
```

**strcpy не проверяет,
хватит ли места в массиве.**

Если памяти недостаточно,
произойдёт выход за границы массива.

Копирование части строки

```
char *strncpy(char *dest,  
              const char *src,  
              size_t n);
```

Копирует не более `n` символов.

Пример strncpy

```
char src[] = "Programming";  
char dst[20];  
  
strncpy(dst, src, 7);  
  
dst[7] = '\\0';  
  
puts(dst);
```

Program

Объединение строк

```
char *strcat(char *dest,  
             const char *src);
```

Добавляет строку

src

в конец строки

dest

Пример strcat

```
char str[32] = "Hello";  
  
strcat(str, ", ");  
strcat(str, "world!");  
  
puts(str);
```

Hello, world!

Как работает strcat?

До вызова

dest

H e l l o \0

src

W o r l d \0

После вызова

dest

H e l l o W o r l d \0

Новые символы записываются вместо символа `'\0'`.

Опасность strcat

```
char str[8] = "Hello";  
strcat(str, " world");
```

**strcat не проверяет,
хватит ли места в массиве.**

Перед объединением необходимо убедиться,
что памяти достаточно.

Добавление части строки

```
char *strncat(char *dest,  
              const char *src,  
              size_t n);
```

Добавляет не более `n` символов из строки `src` .

Сравнение строк

Пусть есть две строки.

```
char a[] = "Hello";  
char b[] = "Hello";
```

Можно ли написать

```
if (a == b)
```

Типичная ошибка

```
char a[] = "Hello";  
char b[] = "Hello";  
  
if (a == b)  
    puts("equal");
```

**Оператор `==` сравнивает адреса,
а не содержимое строк.**

Правильное сравнение

```
int strcmp(const char *s1,  
           const char *s2);
```

Функция сравнивает строки посимвольно.

Пример strcmp

```
char a[] = "Hello";  
char b[] = "Hello";  
  
if (strcmp(a, b) == 0)  
    puts("equal");
```

equal

Результат strcmp

Возвращаемое значение	Значение
< 0	первая строка меньше второй
0	строки равны
> 0	первая строка больше второй

Почему строки можно сравнивать?

Строки сравниваются посимвольно.

```
apple
```

```
apply
```

Сначала сравниваются

```
a = a
```

```
p = p
```

```
p = p
```

```
l = l
```

```
e < y
```

Следовательно

```
"apple" < "apply"
```

Сравнение части строки

```
int strncmp(const char *s1,  
            const char *s2,  
            size_t n);
```

Сравнивает только первые **n** символов.

Пример strcmp

```
char a[] = "Programming";  
char b[] = "Programmer";  
  
if (strcmp(a, b, 7) == 0)  
    puts("equal");
```

equal

Поиск символа

```
char *strchr(const char *str,  
             int ch);
```

Возвращает указатель
на первое вхождение символа.

Если символ не найден,
возвращает `NULL`.

Пример strchr

```
char str[] = "Programming";  
char *p = strchr(str, 'g');  
printf("%td\n", p - str);
```

3

Поиск подстроки

```
char *strstr(const char *str,  
             const char *sub);
```

Ищет первое вхождение одной строки
в другую.

Пример strstr

```
char str[] = "Visual C++";  
char *p = strstr(str, "C++");  
puts(p);
```

C++

Основные функции string.h

Функция	Назначение
<code>strlen</code>	длина строки
<code>strcpy</code>	копирование
<code>strncpy</code>	копирование части строки
<code>strcat</code>	объединение строк
<code>strncat</code>	объединение части строки
<code>strcmp</code>	сравнение строк
<code>strncmp</code>	сравнение части строки
<code>strchr</code>	поиск символа
<code>strstr</code>	поиск подстроки

Типичные ошибки

Большинство ошибок при работе со строками связано с тем, что строка — это обычный массив символов.

Рассмотрим самые распространённые ситуации.

Ошибка №1

Что выведет программа?

```
char a[] = "Hello";  
char b[] = "Hello";  
  
if (a == b)  
    puts("equal");  
else  
    puts("not equal");
```

Ответ

```
char a[] = "Hello";  
char b[] = "Hello";  
  
if (a == b)
```

Оператор

```
==
```

сравнивает адреса массивов, а не их содержимое.

Для сравнения строк используйте `strcmp()` .

Ошибка №2

```
char str[5] = "Hello";
```

Что здесь не так?

Ответ

Строка `hello` занимает шесть символов.

```
h e l l o \0
```

Следовательно,

```
char str[5];
```

недостаточно.

Всегда оставляйте место для символа `'\0'`.

Ошибка №3

```
char str[8];  
  
strcpy(str,  
        "Programming");
```

Что произойдёт?

Ответ

Функция

```
strcpy()
```

копирует всю строку целиком.

Если массив слишком маленький,

копирование продолжится за его пределами.

Это приводит к выходу за границы массива.

Ошибка №4

```
char str[8] = "Hello";  
strcat(str,  
        " world");
```

Что произойдёт?

Ответ

После объединения строк

```
Hello world
```

не помещается в массив.

`strcat()` также не проверяет размер массива.

Ошибка №5

```
const char *str = "Hello";  
str[0] = 'h';
```

Можно ли изменить строку?

Ответ

Строковый литерал является неизменяемым.

Поэтому

```
str[0] = 'h';
```

недопустимо.

Если необходимо изменять символы, используйте

```
char str[] = "Hello";
```

Ошибка №6

```
char src[] = "Programming";  
char dst[16];  
  
strncpy(dst, src, 7);  
  
puts(dst);
```

Всегда ли программа работает правильно?

Ответ

`strncpy()` может не добавить символ

```
'\0'
```

в конец строки.

После копирования обычно необходимо сделать

```
dst[7] = '\0';
```

Не считайте, что `strncpy()` всегда завершает строку.

Ошибка №7

```
char str[] = "Hello";  
  
printf("%zu\n",  
       sizeof(str));  
  
printf("%zu\n",  
       strlen(str));
```

Одинаковые ли значения будут выведены?

Ответ

```
sizeof(str)
```

возвращает размер массива.

```
6
```

```
strlen(str)
```

возвращает длину строки.

```
5
```

sizeof() и **strlen()** — это разные операции.

Зачем нужны файлы?

Во многих программах данные необходимо сохранять между запусками.

Например:

- настройки программы;
- список пользователей;
- результаты вычислений;
- текстовые документы.

Файлы позволяют хранить информацию на диске компьютера.

Почему недостаточно scanf()?

Программа может считать данные с клавиатуры.

```
char name[64];  
  
scanf("%63s", name);
```

Но после завершения программы
все введённые данные будут потеряны.

Файл позволяет сохранить информацию
для последующих запусков программы.

Что происходит при работе с файлом?



Практически любая работа с файлом состоит из этих трёх этапов.

Потоки

При вводе и выводе данных язык **C** использует понятие **потока** (*stream*).

Поток — это источник данных или место, куда данные отправляются.

Например:

- клавиатура;
- экран;
- файл.

Стандартные потоки

Каждая программа автоматически получает три стандартных потока.

Поток	Назначение
<code>stdin</code>	стандартный ввод
<code>stdout</code>	стандартный вывод
<code>stderr</code>	вывод сообщений об ошибках

Потоки уже знакомы

Когда мы писали

```
scanf("%d", &x);
```

использовался поток `stdin`

Когда писали

```
printf("%d\n", x);
```

использовался поток `stdout`

Файл — это ещё один поток данных.

Тип FILE

Для работы с файлами используется тип

```
FILE
```

Обычно программы работают с указателем на него.

```
FILE *file;
```

Переменная хранит информацию об открытом файле.

Открытие файла

```
FILE *fopen(const char *filename,  
            const char *mode);
```

Функция открывает файл
и возвращает указатель типа

```
FILE *
```

Пример открытия файла

```
FILE *file =  
    fopen("input.txt", "r");
```

После открытия файла
через переменную `file`
можно выполнять чтение и запись.

Режимы открытия

Режим	Назначение
"r"	чтение
"w"	запись
"a"	добавление в конец файла
"rb"	чтение бинарного файла
"wb"	запись бинарного файла

Если файл не открылся

Открытие файла может завершиться ошибкой.

Например:

- файл не существует;
- недостаточно прав доступа;
- ошибка файловой системы.

Проверка результата fopen()

```
FILE *file =  
    fopen("input.txt", "r");  
  
if (file == NULL)  
{  
    printf("Ошибка открытия файла\n");  
}
```

**Всегда проверяйте,
что файл действительно открылся.**

Заккрытие файла

После окончания работы файл необходимо закрыть.

```
fclose(file);
```

После этого операционная система освободит все связанные ресурсы.

Полный пример

```
FILE *file =  
    fopen("input.txt", "r");  
  
if (file == NULL)  
{  
    return 1;  
}  
  
/* работа с файлом */  
  
fclose(file);
```

Чтение строки из файла

```
char *fgets(char *str,  
            int size,  
            FILE *file);
```

Читает одну строку из файла
в массив `str`.

Функция читает не более `size - 1` символов.

Пример fgets

```
char line[128];  
  
fgets(line, sizeof(line), file);  
  
puts(line);
```

Если файл содержит

```
Hello, world!
```

то программа выведет

```
Hello, world!
```

Как работает fgets?

Файл

```
Hello  
Programming  
C language
```

После вызова

```
fgets(line, sizeof(line), file);
```

в массиве окажется

```
Hello\n
```

Следующий вызов прочитает следующую строку.

Чтение всего файла

```
char line[256];

while (fgets(line,
             sizeof(line),
             file) != NULL)
{
    printf("%s", line);
}
```

Каждый вызов `fgets()`

читает следующую строку файла.

Запись строки

```
int fputs(const char *str,  
          FILE *file);
```

Записывает строку в файл.

Пример fputs

```
FILE *file =  
    fopen("output.txt", "w");  
  
fputs("Hello\n", file);  
fputs("Programming\n", file);  
  
fclose(file);
```

Содержимое файла:

```
Hello  
Programming
```

puts() и fputs()

```
puts(str);
```

Записывает строку в

```
stdout
```

```
fputs(str, file);
```

Записывает строку в указанный файл.

Чтение символов

```
int fgetc(FILE *file);
```

Читает один символ из файла.

Возвращает

- код символа;
- либо EOF, если файл закончился.

Пример fgetc

```
int c;  
  
c = fgetc(file);  
  
printf("%c\n", c);
```

Если файл начинается с

Programming

будет выведено

P

Запись символов

```
int fputc(int ch,  
          FILE *file);
```

Записывает один символ в файл.

Пример fputc

```
fputc('A', file);  
fputc('\n', file);  
fputc('B', file);
```

Файл станет содержать

```
A  
B
```

Конец файла

Когда данные заканчиваются, функции чтения сообщают об этом специальным значением

EOF

(End Of File).

Чтение до конца файла

```
int c;  
  
while ((c = fgetc(file))  
       != EOF)  
{  
    putchar(c);  
}
```

**Цикл продолжается,
пока не встретится конец файла.**

Форматированный ввод

```
int fscanf(FILE *file,  
           const char *format,  
           ...);
```

Работает аналогично

```
scanf()
```

но читает данные из файла.

Пример fscanf

Программа

```
int a, b;  
  
fscanf(file,  
        "%d %d",  
        &a, &b);
```

Файл

10 25

После выполнения

a = 10

b = 25

Форматированный вывод

```
int fprintf(FILE *file,  
            const char *format,  
            ...);
```

Работает аналогично

```
printf()
```

но записывает данные в файл.

Пример fprintf

```
int x = 25;  
  
fprintf(file,  
        "x = %d\n",  
        x);
```

Файл будет содержать

```
x = 25
```

Аналоги знакомых функций

Консоль	Файл
<code>scanf()</code>	<code>fscanf()</code>
<code>printf()</code>	<code>fprintf()</code>
<code>gets()</code> *	<code>fgets()</code>
<code>puts()</code>	<code>fputs()</code>
<code>getchar()</code>	<code>fgetc()</code>
<code>putchar()</code>	<code>fputc()</code>

*Функция `gets()` удалена из стандарта **C** и использовать её нельзя.

Перемещение по файлу

```
int fseek(FILE *file,  
          long offset,  
          int origin);
```

Позволяет изменить текущую позицию в файле.

Откуда отсчитывается смещение?

Константа	Начало отсчёта
<code>SEEK_SET</code>	начало файла
<code>SEEK_CUR</code>	текущая позиция
<code>SEEK_END</code>	конец файла

Пример fseek

```
fseek(file, 10, SEEK_SET);
```

0 10

↓ ↓

▼ ▼

Hello, world!

 ↑

После вызова чтение начнётся с десятого байта файла.

Текущая позиция

```
long ftell(FILE *file);
```

Возвращает номер текущего байта в файле.

```
printf("%ld\n", ftell(file));
```

Возврат в начало файла

```
rewind(file);
```

Эквивалентно

```
fseek(file, 0, SEEK_SET);
```

После вызова чтение снова начнётся с начала файла.

Текстовые и бинарные файлы

Текстовый файл

```
123  
Hello
```

можно открыть обычным текстовым редактором.

Бинарный файл содержит произвольные байты и предназначен для чтения программами.

Чтение бинарных данных

```
size_t fread(void *ptr,  
             size_t size,  
             size_t count,  
             FILE *file);
```

Читает данные из файла
не как текст,
а как последовательность байтов.

Пример fread

```
int numbers[100];  
  
fread(numbers,  
    sizeof(int),  
    100,  
    file);
```

Будет прочитано до 100 целых чисел.

Запись бинарных данных

```
size_t fwrite(const void *ptr,  
              size_t size,  
              size_t count,  
              FILE *file);
```

Записывает данные в файл в виде последовательности байтов.

Пример fwrite

```
int numbers[100];  
  
fwrite(numbers,  
        sizeof(int),  
        100,  
        file);
```

Массив будет записан в бинарный файл.

Ошибка №1

```
FILE *file =  
    fopen("input.txt", "r");  
  
fgets(buf,  
    sizeof(buf),  
    file);
```

Что здесь не так?

Ответ

Не проверяется результат

```
fopen()
```

Если файл не открылся,

```
file == NULL
```

и обращение к нему приведёт к ошибке.

Всегда проверяйте результат `fopen()` .

Ошибка №2

```
while (!feof(file))  
{  
    fgets(buf,  
          sizeof(buf),  
          file);  
  
    puts(buf);  
}
```

Работает ли такой цикл правильно?

Ответ

Нет.

Функция

```
feof()
```

становится истинной только после неудачной попытки чтения.

В результате последняя строка может быть обработана дважды.

Правильный вариант

```
while (fgets(buf,  
             sizeof(buf),  
             file) != NULL)  
{  
    puts(buf);  
}
```

Проверять нужно результат функции чтения, а не признак конца файла.

Ошибка №3

```
FILE *file =  
    fopen("output.txt", "w");  
  
/* работа с файлом */  
  
return 0;
```

Что забыли сделать?

Ответ

После окончания работы необходимо закрыть файл.

```
fclose(file);
```

Всегда закрывайте открытые файлы.

Основные функции работы с файлами

Назначение	Функция
открыть файл	<code>fopen</code>
закрыть файл	<code>fclose</code>
чтение строки	<code>fgets</code>
запись строки	<code>fputs</code>
чтение символа	<code>fgetc</code>
запись символа	<code>fputc</code>
форматированное чтение	<code>fscanf</code>

Назначение	Функция
форматированная запись	<code>fprintf</code>
чтение бинарных данных	<code>fread</code>
запись бинарных данных	<code>fwrite</code>
изменить позицию	<code>fseek</code>
узнать позицию	<code>ftell</code>
перейти в начало	<code>rewind</code>